

P1746R0

Feedback on [P1386R2] std::audio

Published Proposal, 2019-06-14

This version:

<http://wg21.link/P1746R0>

Authors:

[Richard Powell](#) (Apple)

[David Singer](#) (Apple)

[Sophia Poirier](#) (Apple)

[Dan Klingler](#) (Apple)

[Tony Guetta](#) (Apple)

[JF Bastien](#) (Apple)

Audience:

SG13, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Abstract

2 Audio Device Interfaces

2.1 Real-time Device Timing Constraints

2.2 Audio Routing and Policies

2.3 Audio Formats

2.4 Audio Device

2.5 Portability

3 What we should standardize

3.1 Audio Data

3.2 Algorithms

4 Conclusion

References

Informative References

§ 1. Abstract

We have gathered input from a variety of folks involved in audio at Apple, and here is our joint, considered position regarding the `std::audio` proposal in [\[P1386R2\]](#).

Audio is important to the Apple ecosystem. The type system and determinism of C++ lends itself well to the audio software domain. In the proposal we like the formalization of data types and algorithms that are common in the audio domain. However, we are concerned about the audio device interfaces and requiring C++ systems to have a specific implementation.

§ 2. Audio Device Interfaces

Creating a good interface between software and audio hardware is something that on the surface seems straight-forward, but on a practical system is challenging to implement correctly. The design in [\[P1386R2\]](#) does not address several important areas.

§ 2.1. Real-time Device Timing Constraints

Software systems that interface with audio hardware requires very special consideration because of the nature of real-time systems and hard deadlines. Audio hardware is particularly challenging because incorrect implementations can cause unpredictable performance problems or worse, may damage audio reproduction hardware. This can arise when audio data is not returned to the system in time. The outcome of such an event usually an audible glitch due to the introduction of a gap of silence into the audio stream, potentially producing dramatic waveform discontinuities that can damage speaker cones.

In the paper's design proposal, the `audio::device` runs by repeatedly calling a user supplied callback with request for more audio data (or on recording audio, buffers that contain audio data). The implied contract is that the callback needs to complete execution within the time period represented by the audio. In order to do so "no operations that could possibly block for an indeterminate length of time". However, what this is does not have a clear definition, nor are the consequence of violating this

requirement. Incorporating the SG1 work on forward progress guarantees may improve this proposal. The SG1 work on signal safe handlers may be relevant example of addressing this sort of topic as well.

If the committee was to attempt to standardize the `audio::device` callback constraint, we would quickly find this challenging. Would any atomic operation be considered a "blocking operation"? Memory reads can often cause paging or disk reads, so would reading any variable be considered a "blocking operation"?

Beyond the challenge of defining what operations are allowable within the callback, what would be the result of violating the callback? If violation of this time constraint results in Undefined Behavior, would it not be incorrect of any implementation that could detect you would make a blocking call and "optimize" away the operation?

§ 2.2. Audio Routing and Policies

Audio data routing is a challenging area for any program that implements audio. Modern systems have many different audio destinations and sources, from built-in speakers and microphones, to plug and play devices that may be added or removed at any time during a program's lifetime. Audio is a "shared" resource on the system, so coordinating several different programs' access to hardware and what the policy should be (mixing, interrupting, blocking) is necessary. Access to audio hardware is also a very sensitive privacy topic -- software that has unconstrained access to microphones can be a serious concern for modern systems.

An audio interface design that does not take into consideration audio routing and policies will not be an interface upon which large scale portable systems can be created. A C++ developer who uses a simplified interface will likely constantly hit questions like "where did my audio go" or "why can't I hear anything" without being able to determine what the issue is. Security and privacy policies that strengthen implementations over time may break existing C++ implementations, or worse, may constrain systems from making changes in policies.

Many audio devices can dynamically become unavailable while in use or newly available at any time. The paper describes an API for iterating available audio devices at a point in time but does not address dynamic device availability changes. What happens to a client with an open session to a device that becomes unavailable? Is there a means for the client to be notified when a device newly becomes available, or any availability change occurs to the device list? What does a "default device" mean in a system where the underlying device fulfilling that role may change dynamically? If a client creates a session with the default device and that role changes to a different underlying device during the client session, what happens?

The latest revision includes comments about audio sessions, policies, and focus on mobile platforms concludes that such "use cases" are "highly platform-specific" and "therefore out of scope for this proposal". On iOS and watchOS, this does not describe "use cases", it describes the platform. There is no use case for audio on iOS that does not contend with audio sessions, policies, and focus. Does the proposal therefore contend that iOS support is out of scope? This contradicts the stated goal to offer cross-platform audio functionality for hardware including phones.

It is indeed difficult to reconcile the current API with the audio system of a mobile platform like iOS. What is an `audio_device` or `audio_device_list` on iOS? iOS is a platform that does not provide APIs for clients to view anything about what audio devices may or may not be available. However excluding such platforms opens an enormous hole in a cross-platform library intended for standardization.

§ 2.3. Audio Formats

Platform-level support for spatial audio formats with associated positional metadata, and potentially other encoded formats, should be taken into consideration. The current revision of the proposal explicitly declines to provide support for such functionality. We do wish for the domain to continue to be considered, however, as any design should not preclude or obstruct the use of such functionality offered by an operating system.

§ 2.4. Audio Device

The `audio_device::set_buffer_size_frames` API contains critical ambiguities. Is this a request to change the buffer size merely as the client's format via a hardware driver abstraction layer (independent of actual hardware state, potentially involving re-blocking)? Or is this a request to change physical hardware buffer sizes of a device? If the latter, what happens in a multi-client / shared-access scenario? This issue requires consideration about whether this API wishes to support only one category of the functionality or both, and then requires further specification to make those design choices explicit.

Similar concerns that we raised with `audio_device::set_sample_rate` have been partially addressed in the latest revision to the proposal. However it is difficult to envision a workable API for clients where "it is unspecified whether this will cause other clients of the same device to also observe a sample rate change". There is no description of what could be entailed by an unexpected sample rate change for another client. And how does this reconcile with the specification of `audio_device::set_sample_rate` as not changing hardware sample rate but rather resampling where necessary? Why would one client's resampling affect others clients' sample rates? This seems to

indicate that anyone using this `std::audio_device` may have unspecified changes to sample rate occurring in an unspecified manner at unpredictable times due to another client's usage of `std::audio_device`. With those ambiguities, how can developers realistically be expected to rely on this library for stable audio functionality?

Given that `audio_device::set_sample_rate` does not change the physical audio hardware sample rate state, why is `audio_device::get_supported_sample_rates` part of the API? What meaning does it convey to a client if the hardware sample rate is otherwise opaque and client resampling occurs when needed to reconcile rates? Given client resampling provided to match the underlying audio hardware's sample rate, arbitrary client sample rates should be supported.

What is a client to do when `audio_device::stop` returns false? What actionable information does that convey?

§ 2.5. Portability

The proposal leaves us with some broader questions about the goal of being a portable abstraction. We have concern with the proposal's mixed approach to fundamental audio system inconsistencies such as callback (e.g. [\[CoreAudio\]](#) on macOS) vs polling (e.g. WASAPI on Windows) audio rendering APIs. Those are the only two platforms addressed in the proposal and the sample code shows them handled entirely differently. The same goes for the brief mention of audio hardware timestamps and its importance on some platforms but nonexistence on others. Is the proposed API failing to be a general and portable interface? Does it leak too much of the underlying platform specifics?

§ 3. What we should standardize

While interfacing with Audio devices and hardware has a set of challenges described above, dealing with audio data in C++ is something that could greatly benefit from standardization.

§ 3.1. Audio Data

Audio data is frequently transported between systems in a variety of compressed formats that are contained in a plethora of audio transport files and streams (mpeg, etc). However, as the authors of [\[P1386R2\]](#) point out, within a program audio data is almost universally represented as arrays of PCM data values. Having a standardized way to represent this audio would be of great benefit.

However, while the authors' description of interleaved audio is common, frequently de-interleaved audio is not *contiguously* deinterleaved. The audio data representing one channel of audio data may not be located in memory near to the audio data representing the next channel. Frequently the lower level implementation can have de-interleaved channels of data located in different areas of memory, with a unifying data structure that essentially is an array of pointers (AudioDataBufferList reference). This would mean that the `stride_span` data object may not be suitable, and this may impose data copying costs on any implementation.

§ 3.2. Algorithms

Audio signal processing has a number of common algorithms that would benefit greatly from standardization. Here is a (non-exhaustive) list of algorithms that we would recommend pursuing:

- `interleave/deinterleave`: Algorithms to interleave or deinterleave audio data
- `extract_channel`: Extracting a single channel or set of channels from interleaved/deinterleaved data
- `filter`: sets of common filter types, such as FIR or Biquad filter (reference: DSP algorithms)
- `amplify`: perform gain with saturation or clipping
- `sample rate conversion`
- `noise generation`

It should be noted that these algorithms may be constructed by existing STL algorithms like `transform` with specialized lambda functions.

§ 4. Conclusion

We think it is worth exploring the underlying audio data types, containers, and basic algorithms more. These are the foundations of any audio software, whether dealing with audio hardware or not, and building stronger fundamentals in these areas would strengthen this proposal as well as providing value beyond it. For example, as off-line algorithms that process audio data are becoming in more demand, having a solid foundation of types to manipulate this data is essential.

The audio device side of the world, on the other hand, is very complicated. If we were to standardize an audio device interface, we would want to ensure that the implementation has audio routing and policies as a first principle design. This would likely complicate the design, and perhaps not provide the simple interface desired by the authors. But this may tell us that standardizing a simple design for audio is not recommended -- audio I/O is not simple on modern machines. We recognize that

experimentation is part of any TS, but we would hope to see a fuller offering of fundamental audio types and algorithms as the foundation of this proposal.

§ References

§ Informative References

[CoreAudio]

Apple Inc. Core Audio Overview. 2017-10-30. URL: <https://developer.apple.com/library/archive/documentation/MusicAudio/Conceptual/CoreAudioOverview/Introduction/Introduction.html>

[P1386R2]

Guy Somberg; Guy Davidson; Timur Doumler. A Standard Audio API for C++: Motivation, Scope, and Basic Design. 2019. URL: <http://wg21.link/P1386R2>